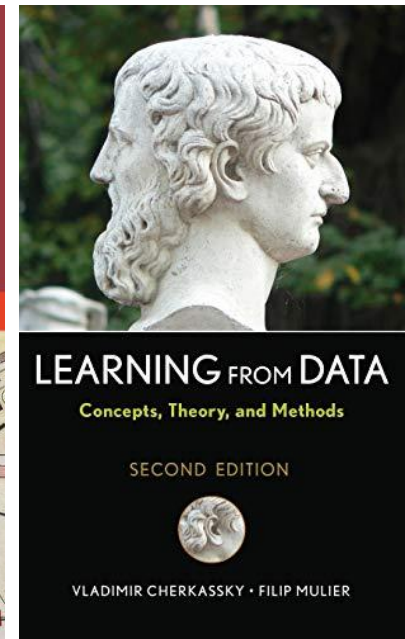
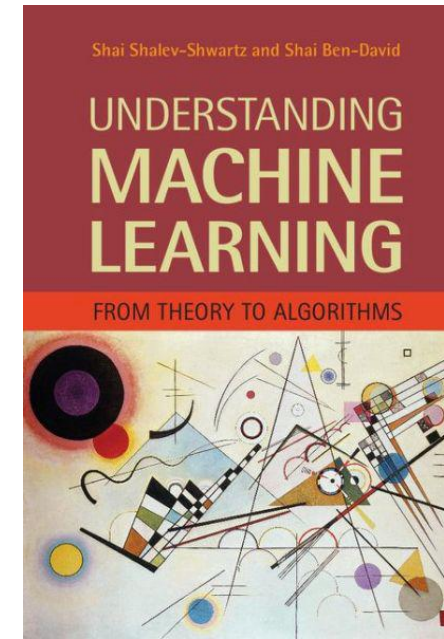
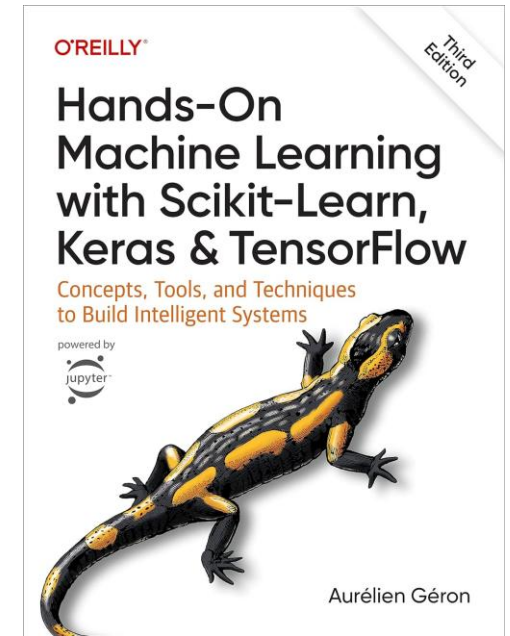


Aprendizaje Automático

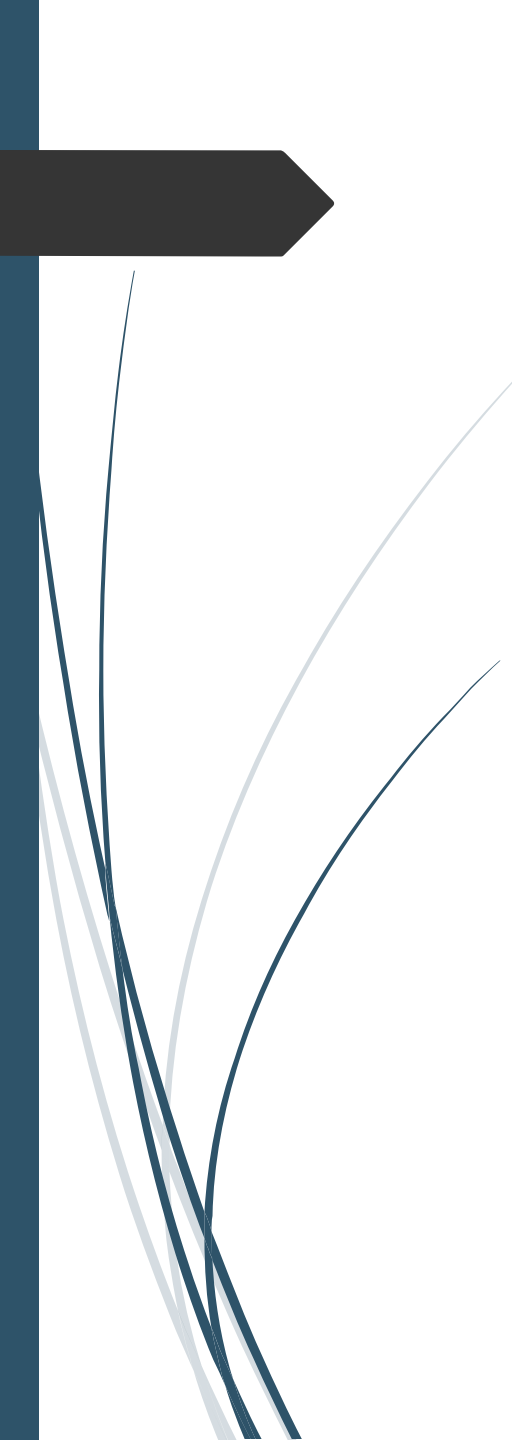
Curso 2025-2026

Tema 5. Aprendizaje Profundo. Fundamentos

Bibliografía

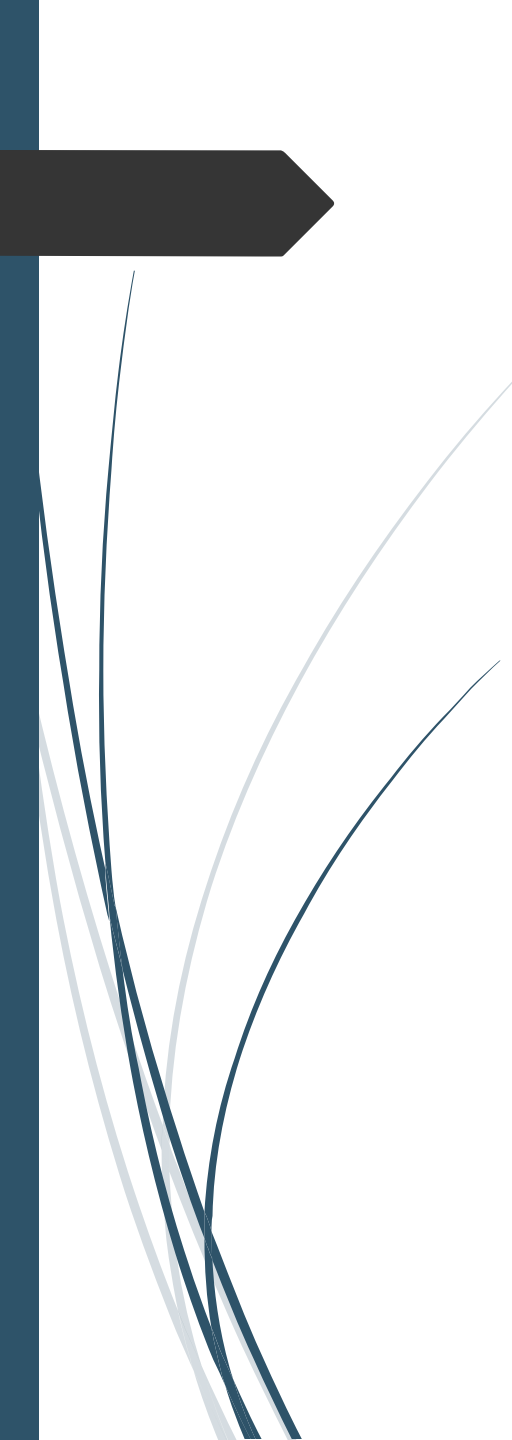


- 
- Tema 1: Conceptos básicos del Aprendizaje Automático
 - Tema 2: Aprendizaje Supervisado. Regresión y Clasificación. Modelos. Evaluación
 - Tema 3: Ingeniería de Características y Preprocesamiento de Datos. Calidad de Datos
 - Tema 4: Aprendizaje no Supervisado
 - **Tema 5: Aprendizaje Profundo. Fundamentos**
 - Tema 6: Aprendizaje Profundo en diferentes tipos de datos y problemas
 - Tema 7. Aprendizaje por refuerzo
 - Tema 8: Optimización en Aprendizaje Automático. Hiperparámetros, selección de modelos y regularización
 - Tema 9: Aspectos Avanzados, retos y desafíos del aprendizaje automático



Redes neuronales artificiales

- Las RNA son el núcleo del aprendizaje profundo.
- Son versátiles, potentes y escalables, por lo que resultan ideales para abordar tareas de aprendizaje automático de gran envergadura y complejidad, como clasificar miles de millones de imágenes (por ejemplo, Google Images), impulsar servicios de reconocimiento de voz (por ejemplo, Siri de Apple), recomendar los mejores vídeos a cientos de millones de usuarios cada día (por ejemplo, YouTube) o aprender a vencer al campeón del mundo de Go (AlphaGo de DeepMind).



De las neuronas biológicas a las artificiales

- Las RNA fueron introducidas por primera vez en 1943 por el neurofisiólogo Warren McCulloch y el matemático Walter Pitts.
- En su artículo "A Logical Calculus of Ideas Immanent in Nervous Activity" (Un cálculo lógico de las ideas inherentes a la actividad nerviosa), presentan un modelo computacional simplificado de cómo las neuronas biológicas podrían trabajar juntas en cerebros animales para realizar cálculos complejos utilizando la lógica proposicional.
- Esta fue la primera arquitectura de red neuronal artificial. Desde entonces se han inventado muchas otras arquitecturas.



De las neuronas biológicas a las artificiales

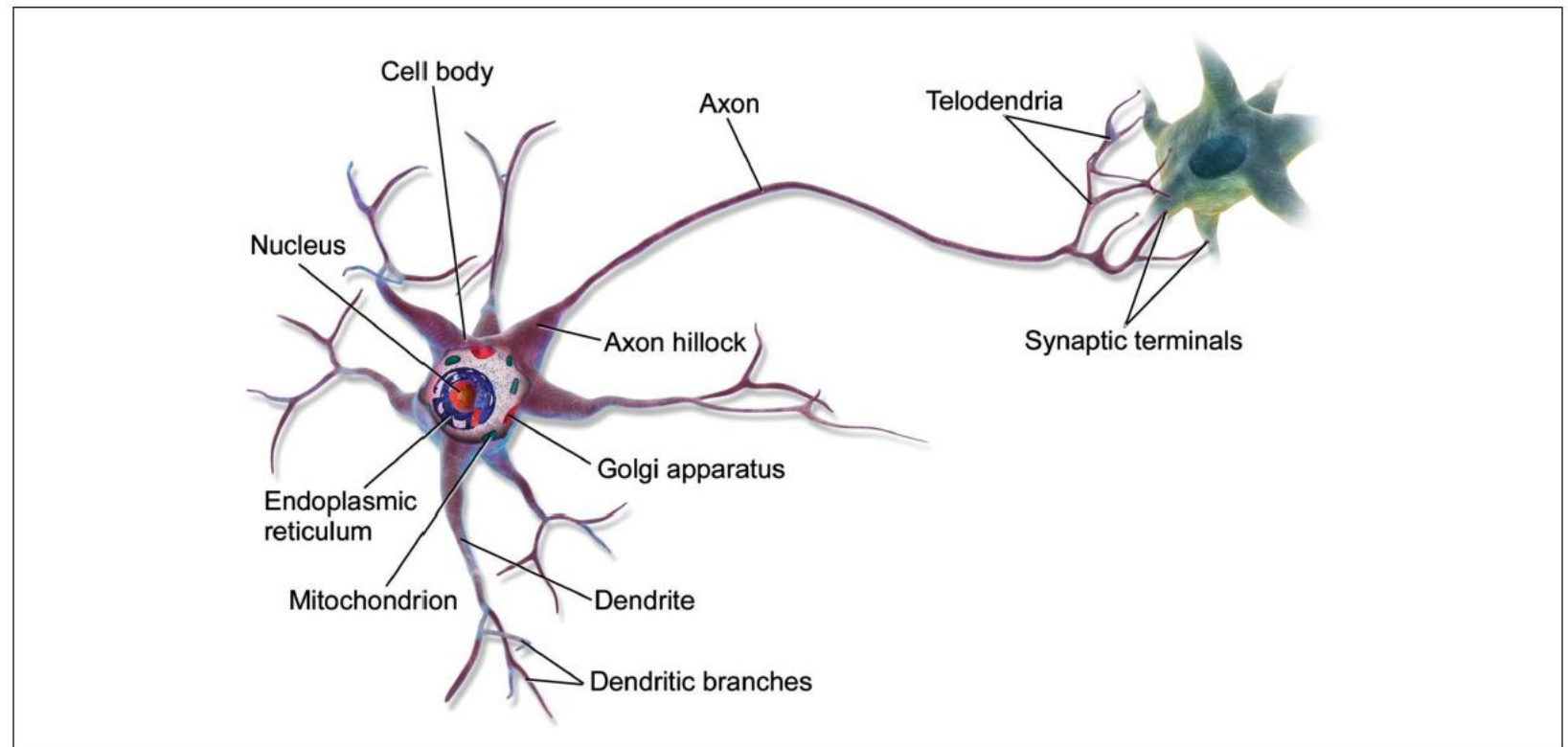
- Los primeros éxitos de las RNA llevaron a la creencia generalizada de que pronto conversaríamos con máquinas verdaderamente inteligentes.
- Cuando en la década de 1960 quedó claro que esta promesa no se cumpliría (al menos durante bastante tiempo), la financiación se fue a otra parte y las RNA entraron en un largo invierno.
- A principios de la década de 1980, se inventaron nuevas arquitecturas y se desarrollaron mejores técnicas de entrenamiento, lo que hizo renacer el interés por el conexionismo, el estudio de las redes neuronales.
- Pero los avances fueron lentos, y en la década de 1990 ya se habían inventado otras potentes técnicas de aprendizaje automático, como las SVM. Estas técnicas parecían ofrecer mejores resultados y fundamentos teóricos más sólidos que las RNA, por lo que una vez más se aparcó el estudio de las redes neuronales.

De las neuronas biológicas a las artificiales

- Estamos asistiendo a otra oleada de interés por las RNA. ¿Se extinguirá esta oleada como las anteriores? He aquí algunas buenas razones para creer que esta vez es diferente y que el renovado interés por las RNA tendrá un impacto mucho más profundo en nuestras vidas:
 - Actualmente se dispone de una enorme cantidad de datos para entrenar redes neuronales, y las RNA superan con frecuencia a otras técnicas de ML en problemas muy grandes y complejos.
 - El enorme aumento de la potencia de cálculo desde los años 90 permite ahora entrenar grandes redes neuronales en un tiempo razonable. Esto se debe en parte a la ley de Moore, pero también gracias a la industria del videojuego, que ha estimulado la producción de potentes tarjetas GPU. Además, las plataformas en la nube han puesto esta potencia al alcance de todos.
 - Se han mejorado los algoritmos de entrenamiento. Para ser justos, sólo difieren ligeramente de los utilizados en los años 90, pero estos ajustes relativamente pequeños han tenido un enorme impacto positivo.
 - Algunas limitaciones teóricas de las RNA han resultado ser benignas en la práctica. Por ejemplo, mucha gente pensaba que los algoritmos de entrenamiento de las RNA estaban condenados al fracaso porque era probable que se atascaran en óptimos locales, pero resulta que esto no es un gran problema en la práctica, sobre todo para las redes neuronales más grandes: los óptimos locales suelen funcionar casi tan bien como el óptimo global.
 - Las RNA parecen haber entrado en un círculo virtuoso de financiación y progreso. Productos asombrosos basados en RNA aparecen periódicamente en los titulares de los periódicos, lo que atrae cada vez más atención y financiación hacia ellos, lo que se traduce en más y más avances y productos aún más asombrosos.

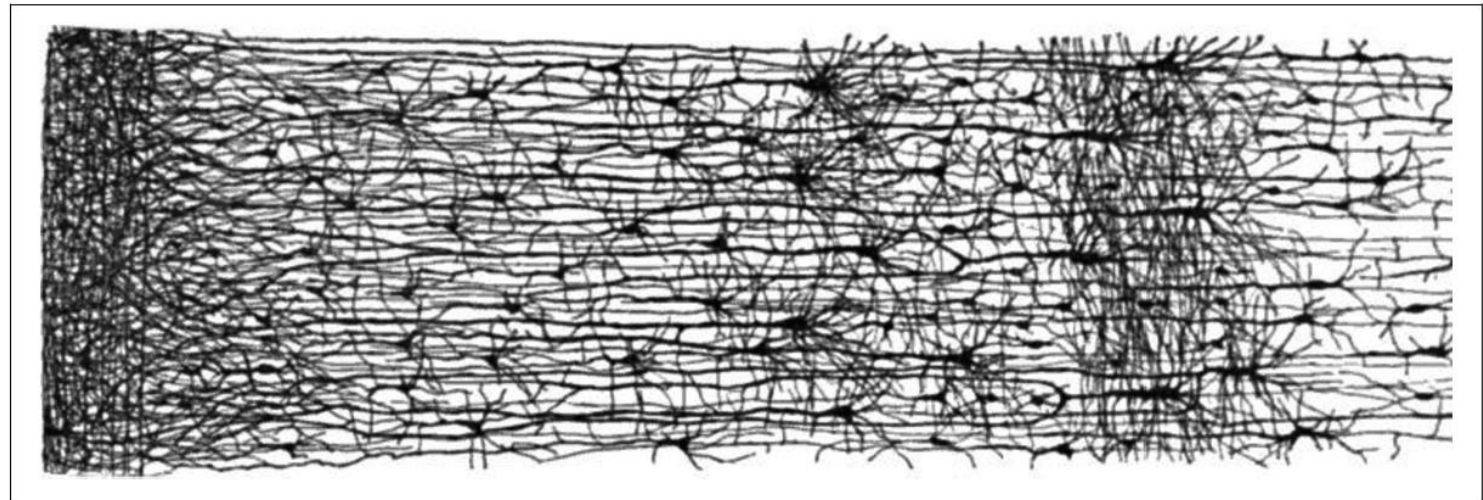
Neuronas biológicas

- Las neuronas biológicas producen breves impulsos eléctricos llamados *potenciales de acción* (PA, o simplemente *señales*), que viajan por los axones y hacen que las sinapsis liberen señales químicas llamadas *neurotransmisores*.



Neuronas biológicas

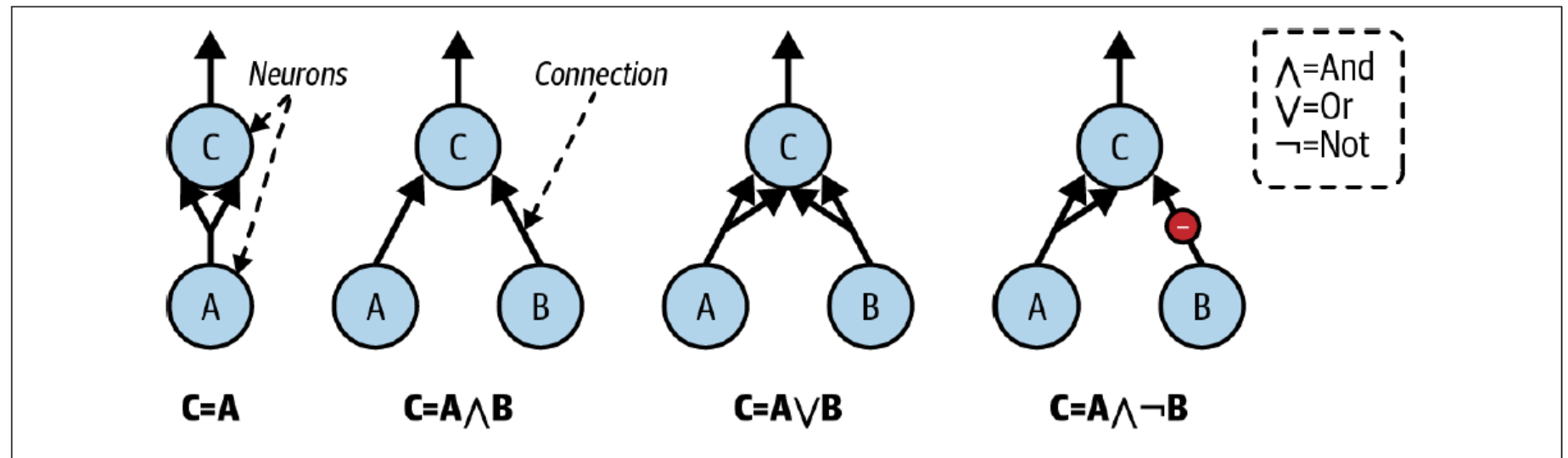
- Así, las neuronas biológicas individuales parecen comportarse de manera simple, pero están organizadas en una vasta red de miles de millones, en la que cada neurona suele estar conectada a miles de otras neuronas. La arquitectura de las redes neuronales biológicas es objeto de investigación activa, pero ya se han cartografiado algunas partes del cerebro. Estos esfuerzos muestran que las neuronas suelen estar organizadas en capas consecutivas, especialmente en la corteza cerebral (la capa externa del cerebro).



Dibujo de una laminación cortical de S. Ramón y Cajal (dominio público)

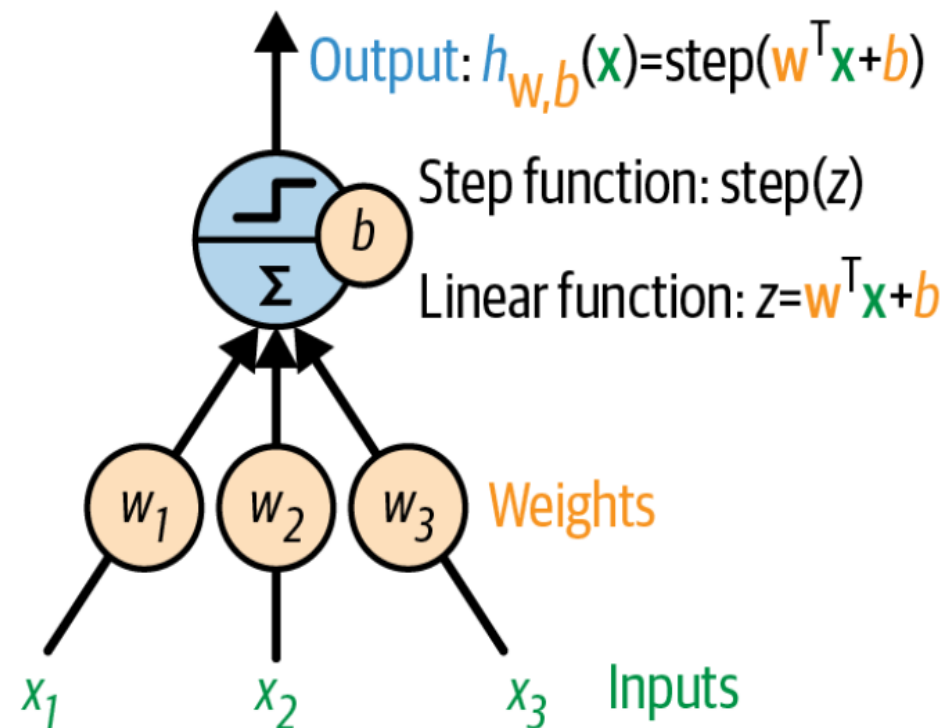
Cálculos lógicos con neuronas

- McCulloch y Pitts propusieron un modelo muy simple de la neurona biológica, que más tarde se conoció como neurona artificial: tiene una o más entradas binarias (on/off) y una salida binaria.
- La neurona artificial activa su salida cuando más de un cierto número de sus entradas están activas. Demostraron que incluso con un modelo tan simplificado es posible construir una red de neuronas artificiales capaz de calcular cualquier proposición lógica que se desee.



El perceptrón

- El *perceptrón* es una de las arquitecturas de RNA más sencillas, inventada en 1957 por Frank Rosenblatt.
- Se basa en una neurona artificial llamada *unidad lógica de umbral* (TLU), o a veces *unidad de umbral lineal* (LTU)
- Las entradas y salidas son números (en lugar de valores binarios on/off), y cada conexión de entrada está asociada a un peso.
- La TLU calcula primero una función lineal de sus entradas $z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = \mathbf{w}^T \mathbf{x} + b$.
- A continuación, aplica una función escalón al resultado $h_{\mathbf{w}}(\mathbf{x}) = \text{step}(z)$.
- Así que es casi como la regresión logística, excepto que utiliza una función escalonada en lugar de la función logística.
- Al igual que en la regresión logística, los parámetros del modelo son las ponderaciones de entrada $\mathbf{w}$ y el término de sesgo b .



El perceptrón

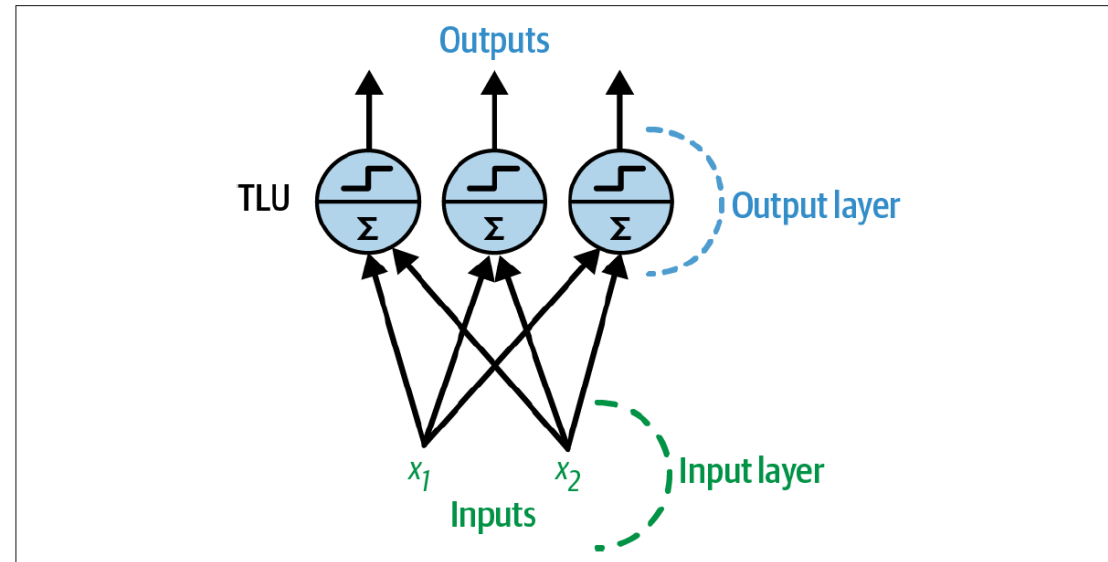
- La función escalón más utilizada en los perceptrones es la *función escalón de Heaviside*. A veces se utiliza en su lugar la función de signo.

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \quad \text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

- Una única TLU puede utilizarse para una clasificación binaria lineal simple.
- Calcula una función lineal de sus entradas y, si el resultado supera un umbral, da como resultado la clase positiva. En caso contrario, emite la clase negativa.
- Esto puede recordarte a la regresión logística o a la clasificación lineal SVM.

El perceptrón

- Un perceptrón se compone de una o más TLU organizadas en una sola capa, donde cada TLU está conectada a cada entrada.
- Una capa de este tipo se denomina *capa totalmente conectada* o *capa densa*. Las entradas constituyen la *capa de entrada*. Y como la capa de TLUs produce las salidas finales, se llama *capa de salida*. Por ejemplo, aquí se representa un perceptrón con dos entradas y tres salidas:



El perceptrón

- Este perceptrón puede clasificar instancias simultáneamente en tres clases binarias diferentes, lo que lo convierte en una clasificación multiclase.
- Gracias a la magia del álgebra lineal, la siguiente ecuación puede utilizarse para calcular eficientemente las salidas de una capa de neuronas artificiales para varias instancias a la vez.

$$h_{\mathbf{W}, \mathbf{b}}(\mathbf{X}) = \phi(\mathbf{XW} + \mathbf{b})$$

- Como siempre, $\mathbf{X}$ representa la matriz de características de entrada. Tiene una fila por instancia y una columna por característica.
- La matriz de pesos $\mathbf{W}$ contiene todos los pesos de conexión. Tiene una fila por entrada y una columna por neurona.
- El vector de sesgo $\mathbf{b}$ contiene todos los términos de sesgo: uno por neurona.
- La función ϕ se llama *función de activación*: cuando las neuronas artificiales son TLUs, es una función escalón (hablaremos de otras funciones de activación en breve).
- Añadir un vector a una matriz significa añadirlo a cada fila de la matriz. ϕ se aplica a cada elemento de la matriz resultante.

El perceptrón

- ¿Cómo se entrena un perceptrón?
- El algoritmo de entrenamiento del perceptrón propuesto por Rosenblatt se inspiró en gran medida en la regla de Hebb. En su libro de 1949 *The Organization of Behavior* (Wiley), Donald Hebb sugirió que cuando una neurona biológica activa a menudo otra neurona, la conexión entre estas dos neuronas se fortalece.
- Los perceptrones se entrenan utilizando una variante de esta regla que tiene en cuenta el error cometido por la red cuando realiza una predicción; la regla de aprendizaje del perceptrón refuerza las conexiones que ayudan a reducir el error.
- Más concretamente, el perceptrón se alimenta de una instancia de entrenamiento cada vez, y para cada instancia hace sus predicciones. Por cada neurona de salida que produce una predicción errónea, refuerza los pesos de conexión de las entradas que habrían contribuido a la predicción correcta. La regla se muestra aquí:

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

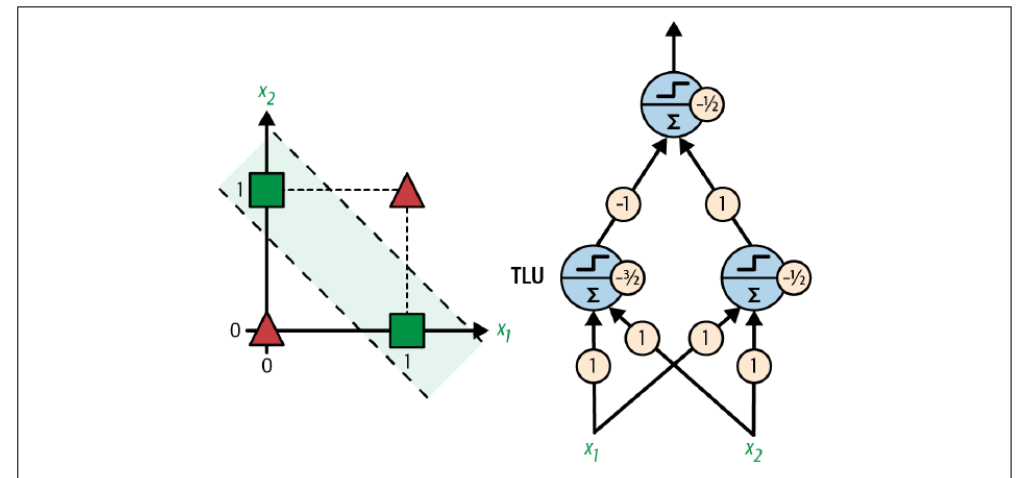
El perceptrón

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

- $w_{i,j}$ es el peso de conexión entre la entrada i^{th} y la neurona j^{th} .
- x_i es el valor de entrada i^{th} de la instancia de entrenamiento actual.
- $\hat{y}_j$ es la salida de la neurona de salida j^{th} para la instancia de entrenamiento actual.
- y_j es la salida objetivo de la neurona de salida j^{th} para la instancia de entrenamiento actual.
- η es la tasa de aprendizaje.
- El límite de decisión de cada neurona de salida es lineal, por lo que los perceptrones son incapaces de aprender patrones complejos (al igual que los clasificadores de regresión logística). Sin embargo, si las instancias de entrenamiento son linealmente separables, Rosenblatt demostró que este algoritmo convergería a una solución.

El perceptrón

- ▶ En su monografía de 1969 Perceptrons, Marvin Minsky y Seymour Papert destacaron una serie de graves deficiencias de los perceptrones, en particular, el hecho de que son incapaces de resolver algunos problemas triviales.
- ▶ Esto es cierto para cualquier otro modelo de clasificación lineal (como los clasificadores de regresión logística), pero los investigadores esperaban mucho más de los perceptrones y algunos se sintieron tan decepcionados que abandonaron las redes neuronales en favor de problemas de más alto nivel como la lógica, la resolución de problemas y la búsqueda. La falta de aplicaciones prácticas tampoco ayudó.

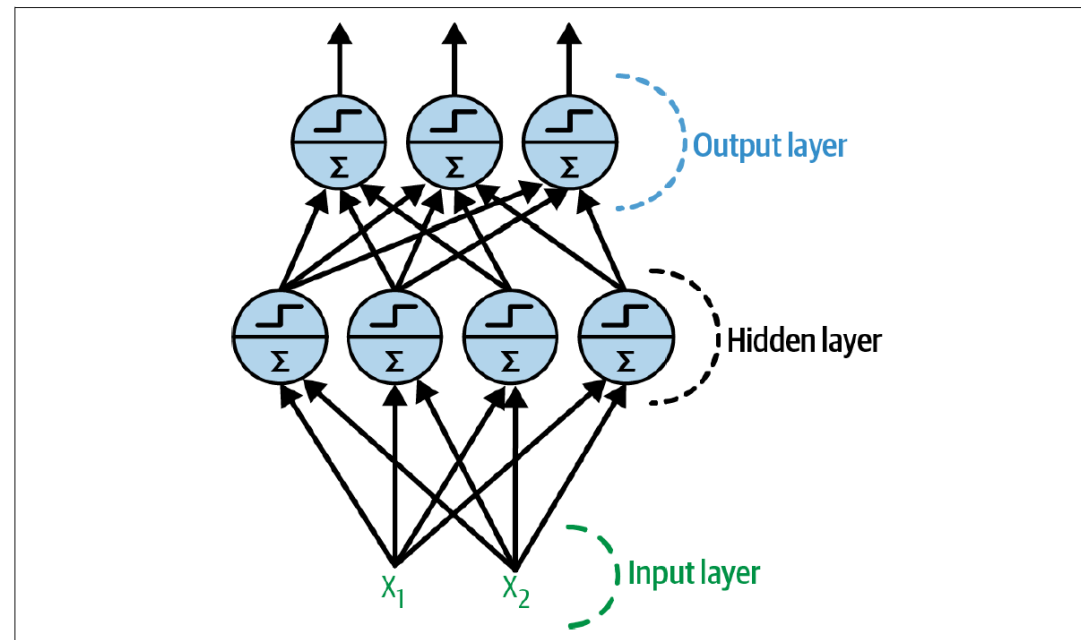


El perceptrón

- Algunas de las limitaciones de los perceptrones pueden eliminarse apilando múltiples perceptrones.
- La RNA resultante se denomina *perceptrón multicapa* (MLP). Un MLP puede resolver el problema XOR, como puedes comprobar calculando la salida del MLP representado en la figura anterior: con entradas (0, 0) o (1, 1), la red da 0, y con entradas (0, 1) o (1, 0) da 1.
 - Por ejemplo, cuando las entradas son (0, 1), la neurona inferior izquierda calcula $0 \times 1 + 1 \times 1 - 3/2 = -1/2$, que es negativo, por lo que su salida es 0. La neurona inferior derecha calcula $0 \times 1 + 1 \times 1 - 1/2 = 1/2$, que es positivo, por lo que su salida es 1. La neurona de salida recibe las salidas de las dos primeras neuronas como entradas, por lo que calcula $0 \times (-1) + 1 \times 1 - 1/2 = 1/2$. La neurona de salida recibe las salidas de las dos primeras neuronas como entradas, por lo que calcula $0 \times (-1) + 1 \times 1 - 1/2 = 1/2$. Esto es positivo, por lo que la salida es 1. Esto es positivo, por lo que su salida es 1.

El perceptrón multicapa y la retropropagación

- Un MLP se compone de una capa de entrada, una o más capas de TLUs llamadas *capas ocultas*, y una capa final de TLUs llamada capa de salida.
- Las capas cercanas a la capa de entrada suelen denominarse *capas inferiores*, y las cercanas a las salidas, *capas superiores*.



El perceptrón multicapa y la retropropagación

- En 1970, un investigador llamado Seppo Linnainmaa introdujo en su tesis de máster una técnica para calcular todos los gradientes de forma automática y eficiente.
- Este algoritmo se denomina ahora *diferenciación automática en modo inverso* (o *auto-diff en modo inverso*).
- En sólo dos pasadas por la red (una hacia delante y otra hacia atrás), es capaz de calcular los gradientes del error de la red neuronal con respecto a cada uno de los parámetros del modelo.

El perceptrón multicapa y la retropropagación

- En otras palabras, puede averiguar cómo debe ajustarse cada peso de conexión y cada sesgo para reducir el error de la red neuronal.
- A continuación, estos gradientes pueden utilizarse para realizar un paso de descenso de gradiente.
- Si se repite este proceso de calcular los gradientes automáticamente y dar un paso de descenso de gradiente, el error de la red neuronal disminuirá gradualmente hasta que finalmente alcance un mínimo.
- Esta combinación de autodifusión en modo inverso y descenso de gradiente se denomina ahora *retropropagación* (o *backprop* para abreviar).



El perceptrón multicapa y la retropropagación

- En realidad, la retropropagación puede aplicarse a todo tipo de grafos computacionales, no sólo a las redes neuronales.
- En 1985, David Rumelhart, Geoffrey Hinton y Ronald Williams publicaron un artículo pionero en el que analizaban cómo la retropropagación permitía a las redes neuronales aprender representaciones internas útiles.
- Sus resultados fueron tan impresionantes que la retropropagación se popularizó rápidamente en este campo. Hoy es, con diferencia, la técnica de entrenamiento de redes neuronales más popular.

El perceptrón multicapa y la retropropagación

► Retropropagación:

- Maneja un mini lote cada vez (por ejemplo, con 32 instancias cada uno), y pasa por todo el conjunto de entrenamiento varias veces. Cada pasada se denomina época.
- Cada minilote entra en la red a través de la capa de entrada. A continuación, el algoritmo calcula la salida de todas las neuronas de la primera capa oculta, para cada instancia del minilote. El resultado se pasa a la siguiente capa, cuya salida se calcula y se pasa a la siguiente capa, y así sucesivamente hasta que obtenemos la salida de la última capa, la capa de salida. Este es el paso hacia delante: es exactamente como hacer predicciones, con la diferencia de que todos los resultados intermedios se conservan, ya que son necesarios para el paso hacia atrás.
- A continuación, el algoritmo mide el error de salida de la red (es decir, utiliza una función de pérdida que compara la salida deseada y la salida real de la red, y devuelve alguna medida del error).

El perceptrón multicapa y la retropropagación

► Retropropagación:

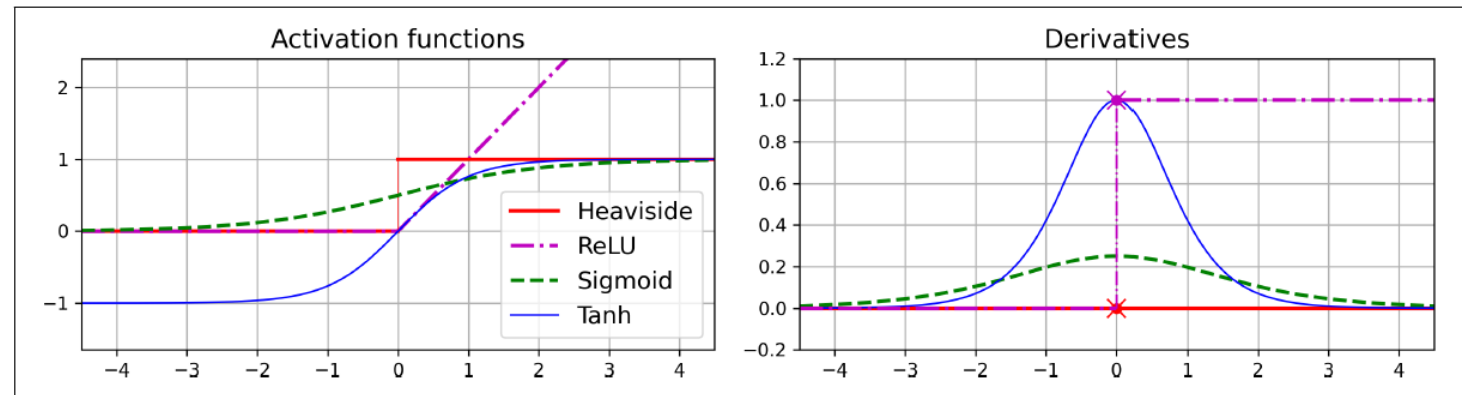
- A continuación, calcula cuánto ha contribuido al error cada sesgo de salida y cada conexión a la capa de salida. Esto se hace analíticamente aplicando la regla de la cadena (quizá la regla más fundamental del cálculo), lo que hace que este paso sea rápido y preciso.
- A continuación, el algoritmo mide qué parte de estas contribuciones de error proceden de cada conexión de la capa inferior, utilizando de nuevo la regla de la cadena y trabajando hacia atrás hasta llegar a la capa de entrada. Como ya se ha explicado, este paso inverso mide eficazmente el gradiente de error en todos los pesos y sesgos de conexión de la red, propagando el gradiente de error hacia atrás a través de la red (de ahí el nombre del algoritmo).
- Por último, el algoritmo realiza un paso de descenso de gradiente para ajustar todos los pesos de conexión de la red, utilizando los gradientes de error que acaba de calcular.

El perceptrón multicapa y la retropropagación

- Es importante inicializar todos los pesos de conexión de las capas ocultas de forma aleatoria, de lo contrario el entrenamiento fallará.
- La retropropagación realiza predicciones para un minilote (paso hacia delante), mide el error, recorre cada capa en sentido inverso para medir la contribución de error de cada parámetro (paso hacia atrás) y, por último, ajusta los pesos y sesgos de conexión para reducir el error (paso de descenso de gradiente).
- Para que el backprop funcionara correctamente, Rumelhart y sus colegas introdujeron un cambio clave en la arquitectura del MLP: sustituyeron la función escalón por la función logística, $\sigma(z) = 1 / (1 + \exp(-z))$, también llamada función *sigmoidea*. Esto era esencial porque la función escalón sólo contiene segmentos planos, por lo que no hay gradiente con el que trabajar (el descenso por gradiente no puede moverse en una superficie plana), mientras que la función sigmoide tiene una derivada no nula bien definida en todas partes, lo que permite al descenso por gradiente progresar en cada paso.

El perceptrón multicapa y la retropropagación: Funciones de activación

- La función tangente hiperbólica: $\tanh(z) = 2\sigma(2z) - 1$
 - Al igual que la función sigmoidea, esta función de activación tiene forma de S, es continua y diferenciable, pero su valor de salida va de -1 a 1 (en lugar de 0 a 1 en el caso de la función sigmoidea). Ese rango tiende a hacer que la salida de cada capa esté más o menos centrada en 0 al principio del entrenamiento, lo que a menudo ayuda a acelerar la convergencia.
- La función lineal unitaria rectificada: $\text{ReLU}(z) = \max(0, z)$
 - La función ReLU es continua pero, por desgracia, no es diferenciable en $z = 0$, y su derivada es 0 para $z < 0$. En la práctica, sin embargo, funciona muy bien y tiene la ventaja de ser rápida de calcular, por lo que se ha convertido en la predeterminada.



MLP de regresión

- Los MLP pueden utilizarse para tareas de regresión. Si quieres predecir un único valor (por ejemplo, el precio de una casa, dadas muchas de sus características), entonces sólo necesitas una única neurona de salida: su salida es el valor predicho.
- Para la regresión multivariante (es decir, para predecir múltiples valores a la vez), se necesita una neurona de salida por dimensión de salida. Por ejemplo, para localizar el centro de un objeto en una imagen, es necesario predecir coordenadas 2D, por lo que se necesitan dos neuronas de salida. Si además quieres colocar un cuadro delimitador alrededor del objeto, entonces necesita dos números más: la anchura y la altura del objeto. Al final, tenemos cuatro neuronas de salida.

Table 10-1. Typical regression MLP architecture

Hyperparameter	Typical value
# hidden layers	Depends on the problem, but typically 1 to 5
# neurons per hidden layer	Depends on the problem, but typically 10 to 100
# output neurons	1 per prediction dimension
Hidden activation	ReLU
Output activation	None, or ReLU/softplus (if positive outputs) or sigmoid/tanh (if bounded outputs)
Loss function	MSE, or Huber if outliers

MLP para Clasificación

- Los MLP también pueden utilizarse para tareas de clasificación. Para un problema de clasificación binaria, sólo se necesita una única neurona de salida que utilice la función de activación sigmoidea: la salida será un número entre 0 y 1, que se puede interpretar como la probabilidad estimada de la clase positiva. La probabilidad estimada de la clase negativa es igual a uno menos ese número.
- Si cada instancia puede pertenecer a una clase, de tres o más clases posibles (por ejemplo, las clases 0 a 9 para la clasificación de imágenes de dígitos), entonces necesita tener una neurona de salida por clase, y debe utilizar la función de activación softmax para toda la capa de salida.
- La función softmax garantizará que todas las probabilidades estimadas estén entre 0 y 1 y que sumen 1, ya que las clases son excluyentes.
- En cuanto a la función de pérdida, dado que estamos prediciendo distribuciones de probabilidad, la pérdida de entropía cruzada (o *entropía x* o pérdida logarítmica para abreviar) suele ser una buena elección.

MLP para Clasificación

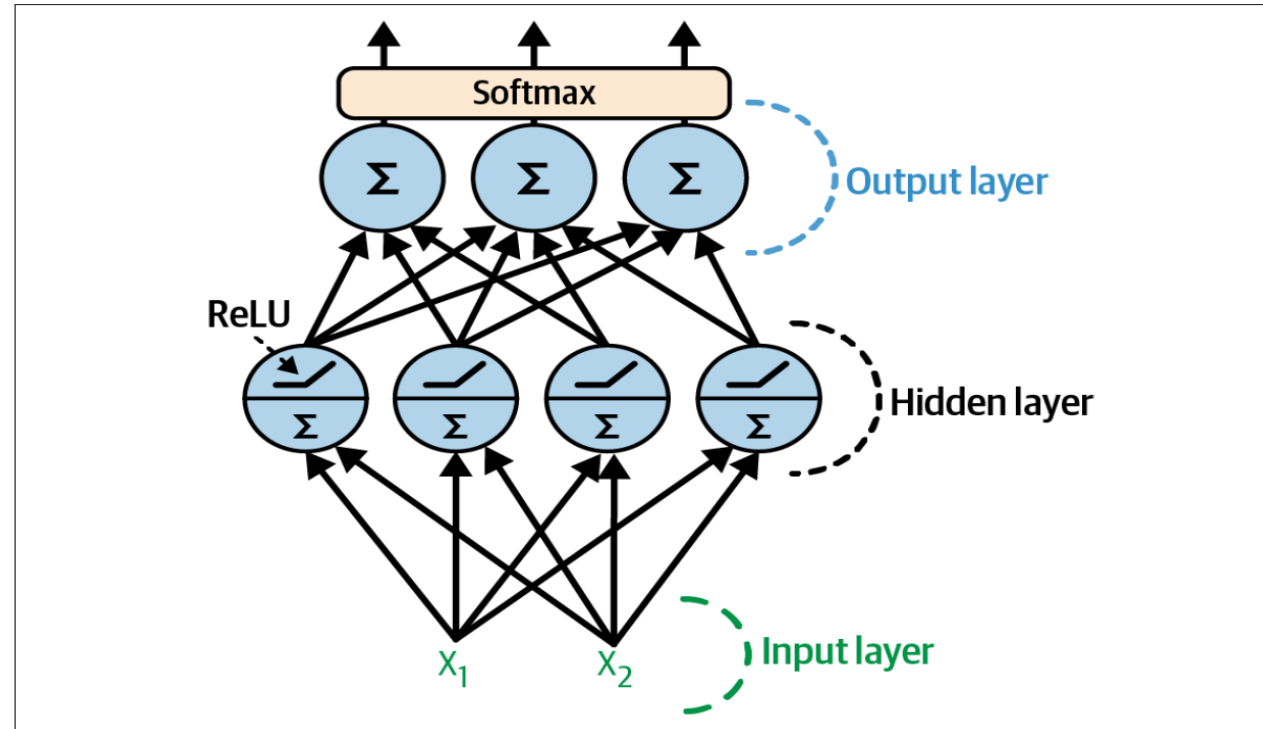


Table 10-2. Typical classification MLP architecture

Hyperparameter	Binary classification	Multilabel binary classification	Multiclass classification
# hidden layers	Typically 1 to 5 layers, depending on the task		
# output neurons	1	1 per binary label	1 per class
Output layer activation	Sigmoid	Sigmoid	Softmax
Loss function	X-entropy	X-entropy	X-entropy

Código 5.1: Ejemplos de RRNNs en Scikit-Learn

Ecosistema de aprendizaje profundo

- Keras es la API de aprendizaje profundo de alto nivel de TensorFlow: permite construir, entrenar, evaluar y ejecutar todo tipo de redes neuronales. La biblioteca Keras original fue desarrollada por François Chollet como parte de un proyecto de investigación y se publicó como proyecto independiente de código abierto en marzo de 2015. Rápidamente ganó popularidad, debido a su facilidad de uso, flexibilidad y bonito diseño.
- Keras solía soportar múltiples backends, incluyendo TensorFlow, PlaidML, Theano, y Microsoft Cognitive Toolkit (CNTK) (los dos últimos están tristemente obsoletos), pero desde la versión 2.4, Keras es sólo TensorFlow. Del mismo modo, TensorFlow solía incluir múltiples APIs de alto nivel, pero Keras fue elegido oficialmente como su API de alto nivel preferida cuando salió TensorFlow 2. La instalación de TensorFlow instalará automáticamente Keras también, y Keras no funcionará sin TensorFlow instalado. En resumen, Keras y TensorFlow se enamoraron y se casaron. Otras bibliotecas populares de aprendizaje profundo son PyTorch de Meta y JAX de Google.

Creación de modelos secuenciales en Keras

- Utilizaremos Fashion MNIST, que es una sustitución directa de MNIST. Tiene exactamente el mismo formato que MNIST (70.000 imágenes en escala de grises de 28×28 píxeles cada una, con 10 clases), pero las imágenes representan artículos de moda en lugar de dígitos escritos a mano, por lo que cada clase es más diversa, y el problema resulta ser significativamente más difícil que MNIST. Por ejemplo, un modelo lineal simple alcanza alrededor del 92% de precisión en MNIST, pero sólo alrededor del 83% en Fashion MNIST.



Creación de modelos secuenciales en Keras

- Ejemplo de modelo secuencial:

```
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.InputLayer(input_shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[28, 28]),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Creación de modelos secuenciales en Keras

► Resumen del modelo:

```
weights, biases = hidden1.get_weights()  
weights
```

```
array([[ 0.02448617, -0.00877795, -0.02189048, ..., -0.02766046,  
        0.03859074, -0.06889391],  
       [ 0.00476504, -0.03105379, -0.0586676 , ...,  0.00602964,  
       -0.02763776, -0.04165364],  
       [-0.06189284, -0.06901957,  0.07102345, ..., -0.04238207,  
        0.07121518, -0.07331658],  
       ...,  
       [-0.03048757,  0.02155137, -0.05400612, ..., -0.00113463,  
        0.00228987,  0.05581069],  
       [ 0.07061854, -0.06960931,  0.07038955, ..., -0.00384101,  
        0.00034875,  0.02878492],  
       [-0.06022581,  0.01577859, -0.02585464, ..., -0.00527829,  
        0.00272203, -0.06793761]], dtype=float32)
```

```
weights.shape
```

```
(784, 300)
```

flatten_input	input:	[(None, 28, 28)]
InputLayer	output:	[(None, 28, 28)]

flatten	input:	(None, 28, 28)
Flatten	output:	(None, 784)

dense	input:	(None, 784)
Dense	output:	(None, 300)

dense_1	input:	(None, 300)
Dense	output:	(None, 100)

dense_2	input:	(None, 100)
Dense	output:	(None, 10)

Creación de modelos secuenciales en Keras

- Compilando el modelo:

```
model.compile(loss="sparse_categorical_crossentropy",  
              optimizer="sgd",  
              metrics=["accuracy"])
```

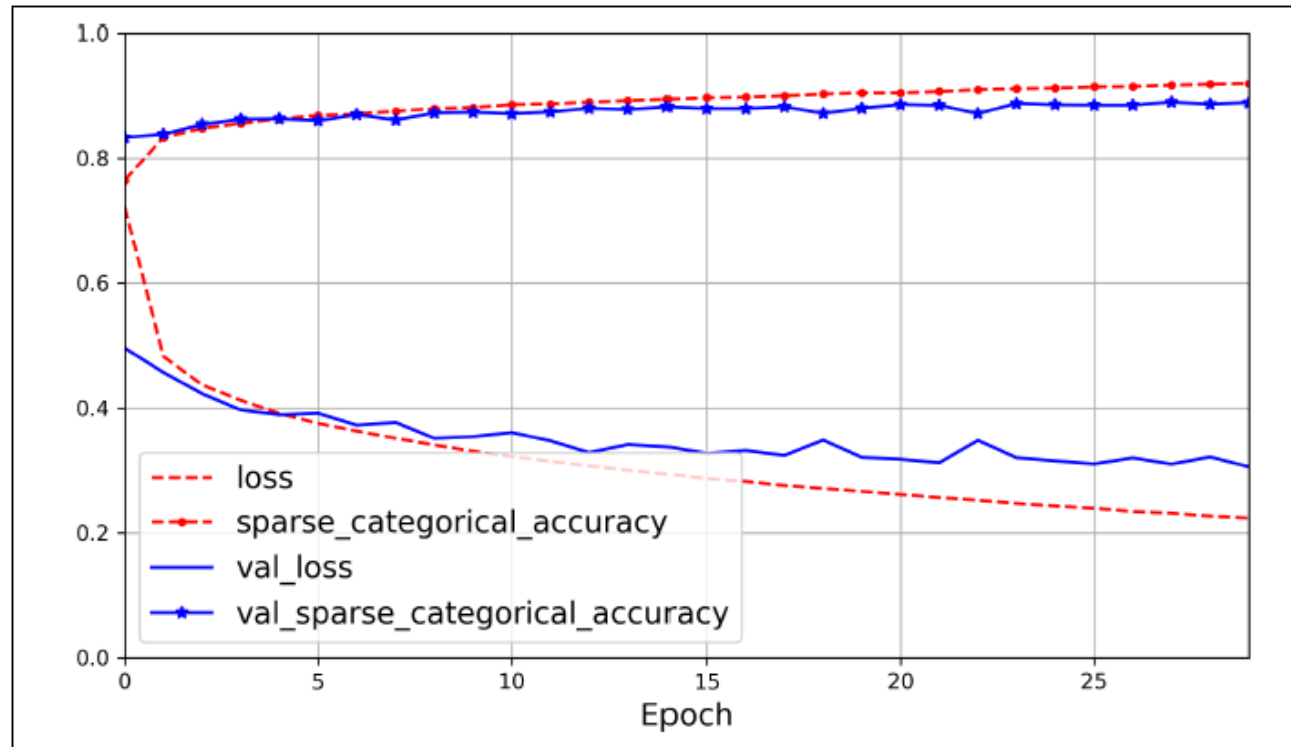
- Entrenar el modelo:

```
history = model.fit(X_train, y_train, epochs=30,  
                   validation_data=(X_valid, y_valid))
```

```
Epoch 1/30  
1719/1719 [=====] - 3s 2ms/step - loss: 0.7221 - sparse_categorical  
accuracy: 0.7647 - val_loss: 0.4961 - val_sparse_categorical_accuracy: 0.8328  
Epoch 2/30  
1719/1719 [=====] - 3s 1ms/step - loss: 0.4826 - sparse_categorical  
accuracy: 0.8332 - val_loss: 0.4563 - val_sparse_categorical_accuracy: 0.8386  
Epoch 3/30  
1719/1719 [=====] - 2s 1ms/step - loss: 0.4370 - sparse_categorical  
accuracy: 0.8476 - val_loss: 0.4232 - val_sparse_categorical_accuracy: 0.8558  
Epoch 4/30  
1719/1719 [=====] - 2s 1ms/step - loss: 0.4123 - sparse_categorical  
accuracy: 0.8561 - val_loss: 0.3971 - val_sparse_categorical_accuracy: 0.8616  
Epoch 5/30
```


Creación de modelos secuenciales en Keras

- Evaluamos el modelo:



```
model.evaluate(X_test, y_test)
```

```
313/313 [=====] - 0s 954us/step - loss: 0.3240 - sparse_categorical_accuracy: 0.8856
```

```
[0.3240019381046295, 0.8855999708175659]
```

Creación de modelos secuenciales en Keras

- Usar el modelo para hacer predicciones:

```
X_new = X_test[:3]  
y_proba = model.predict(X_new)  
y_proba.round(2)
```

1/1 [=====] - 0s 77ms/step

```
array([[0. , 0. , 0. , 0. , 0. , 0.01, 0. , 0.02, 0. , 0.97],  
       [0. , 0. , 0.99, 0. , 0.01, 0. , 0. , 0. , 0. , 0. ],  
       [0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0. , 0. ]],  
      dtype=float32)
```

Ankle boot



Pullover

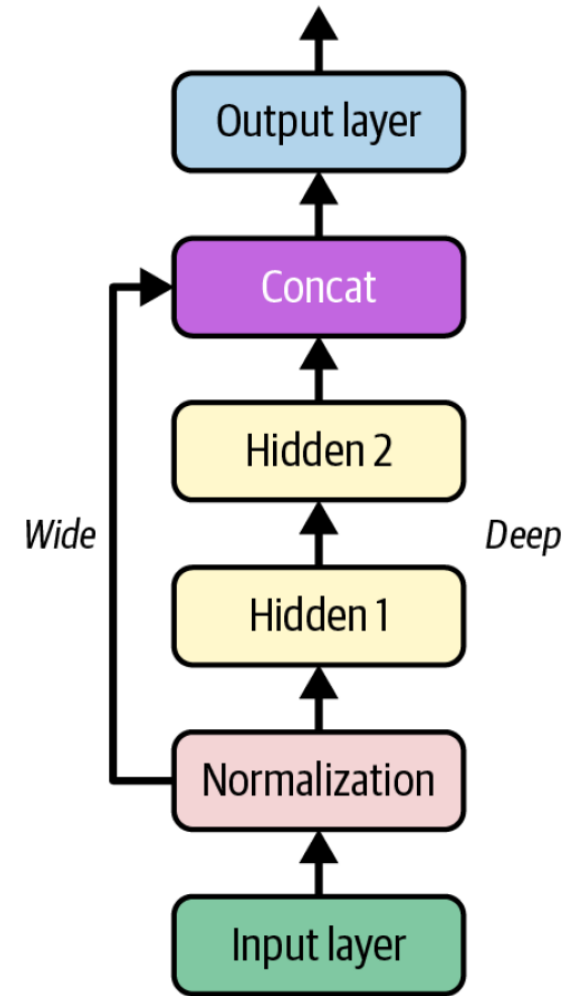


Trouser

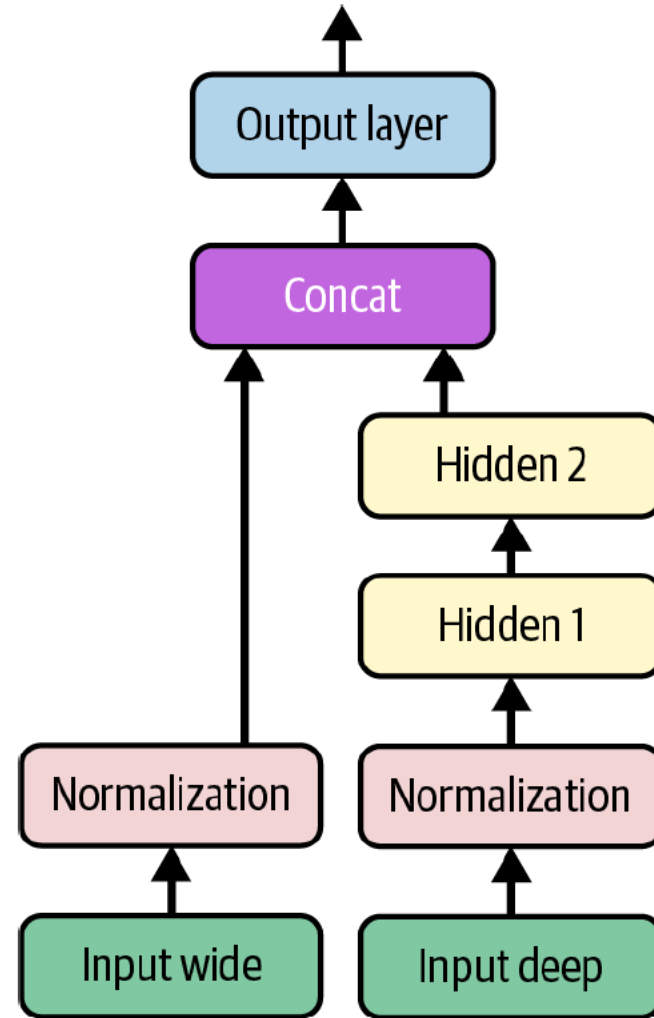


Creación de modelos complejos con la API funcional

- Un ejemplo de red neuronal no secuencial es la red neuronal *Wide & Deep*.
- Esta arquitectura de red neuronal se introdujo en un artículo de 2016 de Heng-Tze Cheng et al.
- Conecta todas o parte de las entradas directamente a la capa de salida, como se muestra aquí.
- Esta arquitectura hace posible que la red neuronal aprenda tanto patrones profundos (mediante la ruta profunda) como reglas sencillas (a través de la ruta corta).
- En cambio, un MLP normal obliga a que todos los datos fluyan a través de toda la pila de capas; así, los patrones simples de los datos pueden acabar distorsionados por esta secuencia de transformaciones.



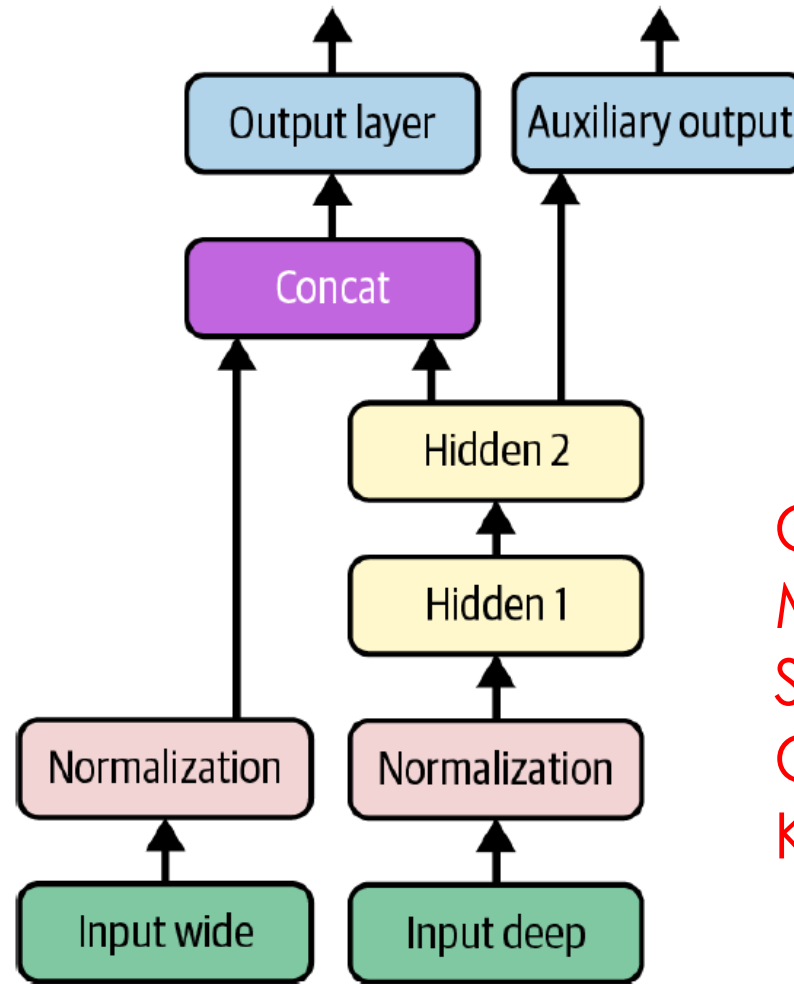
Gestión de entradas múltiples



Manejo de múltiples salidas

- También hay muchos casos de uso en los que es posible que desee tener varias salidas:
 - La tarea puede exigirlo. Por ejemplo, puede que desee localizar y clasificar el objeto principal de una imagen. Se trata tanto de una tarea de regresión como de clasificación.
 - Del mismo modo, puede tener varias tareas independientes basadas en los mismos datos. Claro que podría entrenar una red neuronal por tarea, pero en muchos casos obtendrá mejores resultados en todas las tareas entrenando una sola red neuronal con una salida por tarea. Esto se debe a que la red neuronal puede aprender características de los datos que son útiles en todas las tareas. Por ejemplo, puede realizar una clasificación multitarea de imágenes de caras, utilizando una salida para clasificar la expresión facial de la persona (sonriente, sorprendida, etc.) y otra salida para identificar si lleva gafas o no.
 - Otro caso de uso es como técnica de regularización. Por ejemplo, es posible que desee añadir una salida auxiliar en una arquitectura de red neuronal (véase la siguiente figura) para garantizar que la parte subyacente de la red aprenda algo útil por sí misma, sin depender del resto de la red.

Manejo de múltiples salidas



Código 5.2:
Modelos
Secuenciales y
Complejos en
Keras

Ajuste de los hiperparámetros de las redes neuronales

- La flexibilidad de las redes neuronales es también uno de sus principales inconvenientes: hay muchos hiperparámetros que ajustar. No sólo se puede utilizar cualquier arquitectura de red imaginable, sino que incluso en un MLP básico se puede cambiar el número de capas, el número de neuronas y el tipo de función de activación a utilizar en cada capa, la lógica de inicialización de pesos, el tipo de optimizador a utilizar, su tasa de aprendizaje, el tamaño del lote, y mucho más. ¿Cómo saber qué combinación de hiperparámetros es la mejor para su tarea?
- Una opción es convertir tu modelo Keras en un estimador Scikit-Learn, y luego usar *GridSearchCV* o *RandomizedSearchCV* para ajustar los hiperparámetros. Para ello, puede utilizar las clases *KerasRegressor* y *KerasClassifier* de la biblioteca SciKeras (véase <https://github.com/adriangb/scikeras> para más detalles).
- Sin embargo, hay una forma mejor: puedes utilizar la biblioteca *Keras Tuner*, que es una biblioteca de ajuste de hiperparámetros para modelos Keras.

Ajuste de los hiperparámetros de las redes neuronales

- El ajuste de hiperparámetros sigue siendo un campo de investigación activo, y se están explorando muchos otros enfoques.
- Por ejemplo, echa un vistazo al excelente artículo de DeepMind de 2017, en el que los autores utilizaron un algoritmo evolutivo para optimizar conjuntamente una población de modelos y sus hiperparámetros.
- Google también ha utilizado un enfoque evolutivo, no sólo para buscar hiperparámetros, sino también para explorar todo tipo de arquitecturas de modelos: impulsa su servicio AutoML en Google Vertex AI.
- El término *AutoML* hace referencia a cualquier sistema que se encargue de gran parte del flujo de trabajo de ML. Los algoritmos evolutivos se han utilizado incluso con éxito para entrenar redes neuronales individuales, ¡sustituyendo al omnipresente descenso gradiente!

Código 5.3: Ajuste
Hiperparámetros

Número de capas ocultas

- Para muchos problemas, se puede empezar con una sola capa oculta y obtener resultados razonables. En teoría, un MLP con una sola capa oculta puede modelar incluso las funciones más complejas, siempre que tenga suficientes neuronas. Pero para problemas complejos, las redes profundas tienen una eficiencia de parámetros mucho mayor que las superficiales: pueden modelar funciones complejas utilizando exponencialmente menos neuronas que las redes superficiales, lo que les permite alcanzar un rendimiento mucho mejor con la misma cantidad de datos de entrenamiento.
- Para entender por qué, supongamos que le piden que dibuje un bosque utilizando algún programa de dibujo, pero le prohíben copiar y pegar nada. Le llevaría muchísimo tiempo: tendría que dibujar cada árbol individualmente, rama por rama, hoja por hoja. En cambio, si pudieras dibujar una hoja, copiarla y pegarla para dibujar una rama, luego copiar y pegar esa rama para crear un árbol y, por último, copiar y pegar este árbol para hacer un bosque, habrías terminado en un santiamén. Los datos del mundo real suelen estar estructurados de forma jerárquica, y las redes neuronales profundas lo aprovechan automáticamente.

Número de capas ocultas

- Esta arquitectura jerárquica no sólo ayuda a las DNN a converger más rápidamente hacia una buena solución, sino que también mejora su capacidad de generalización a nuevos conjuntos de datos.
- Por ejemplo, si ya ha entrenado un modelo para reconocer caras en imágenes y ahora quiere entrenar una nueva red neuronal para reconocer peinados, puede iniciar el entrenamiento reutilizando las capas inferiores de la primera red. En lugar de inicializar aleatoriamente los pesos y sesgos de las primeras capas de la nueva red neuronal, puede inicializarlos con los valores de los pesos y sesgos de las capas inferiores de la primera red. De este modo, la red no tendrá que aprender desde cero todas las estructuras de bajo nivel que aparecen en la mayoría de las imágenes; sólo tendrá que aprender las estructuras de alto nivel (por ejemplo, los peinados).
- Esto se denomina *aprendizaje por transferencia*.

Número de neuronas por capa oculta

- El número de neuronas en las capas de entrada y salida viene determinado por el tipo de entrada y salida que requiere la tarea. Por ejemplo, la tarea MNIST requiere $28 \times 28 = 784$ entradas y 10 neuronas de salida.
- En cuanto a las capas ocultas, solía ser común dimensionarlas para formar una pirámide, con cada vez menos neuronas en cada capa, ya que muchas características de bajo nivel pueden fusionarse en muchas menos características de alto nivel. Una red neuronal típica para MNIST podría tener 3 capas ocultas, la primera con 300 neuronas, la segunda con 200 y la tercera con 100.
- Sin embargo, esta práctica se ha abandonado en gran medida porque parece que utilizar el mismo número de neuronas en todas las capas ocultas funciona igual de bien en la mayoría de los casos, o incluso mejor; además, sólo hay que ajustar un hiperparámetro, en lugar de uno por capa. Dicho esto, dependiendo del conjunto de datos, a veces puede ser útil hacer la primera capa oculta más grande que las demás.

Número de neuronas por capa oculta

- Al igual que con el número de capas, puedes probar a aumentar el número de neuronas gradualmente hasta que la red empiece a sobreajustarse.
- Otra posibilidad es construir un modelo con un número de capas y neuronas ligeramente superior al que realmente se necesita y, a continuación, utilizar la parada temprana y otras técnicas de regularización para evitar que se ajuste demasiado.
- Vincent Vanhoucke, científico de Google, lo ha bautizado como el enfoque de los "pantalones elásticos": en lugar de perder el tiempo buscando pantalones que se ajusten perfectamente a tu talla, utiliza pantalones elásticos grandes que se encogerán hasta alcanzar la talla adecuada. Con este planteamiento, se evitan las capas cuello de botella que podrían arruinar el modelo. De hecho, si una capa tiene muy pocas neuronas, no tendrá suficiente poder de representación para conservar toda la información útil de las entradas (por ejemplo, una capa con dos neuronas sólo puede dar salida a datos 2D, por lo que si recibe datos 3D como entrada, se perderá parte de la información). Por muy grande y potente que sea el resto de la red, esa información nunca se recuperará.

Tasa de aprendizaje, tamaño del lote y otros hiperparámetros

► Tasa de aprendizaje

- La tasa de aprendizaje es sin duda el hiperparámetro más importante.
- En general, la tasa de aprendizaje óptima es aproximadamente la mitad de la tasa de aprendizaje máxima.
- Una forma de encontrar una buena tasa de aprendizaje es entrenar el modelo durante unos cientos de iteraciones, empezando con una tasa de aprendizaje muy baja (por ejemplo, 10^{-5}) y aumentándola gradualmente hasta un valor muy grande (por ejemplo, 10). Esto se hace multiplicando la tasa de aprendizaje por un factor constante en cada iteración (por ejemplo, por $(10 / 10^{-5})^{1/500}$ para pasar de 10^{-5} a 10 en 500 iteraciones).
- Si se representa la pérdida en función de la tasa de aprendizaje (utilizando una escala logarítmica para la tasa de aprendizaje), al principio se verá que disminuye. Pero al cabo de un tiempo, la tasa de aprendizaje será demasiado alta, por lo que la pérdida volverá a dispararse: la tasa de aprendizaje óptima será un poco más baja que el punto en el que la pérdida empieza a subir (normalmente, unas 10 veces más baja que el punto de inflexión).
- A continuación, puede reinicializar su modelo y entrenarlo normalmente utilizando este buen ritmo de aprendizaje.



Tasa de aprendizaje, tamaño del lote y otros hiperparámetros

► Optimizador

- También es importante elegir un optimizador mejor que el simple descenso de gradiente por mini lotes (y ajustar sus hiperparámetros).

► Tamaño del lote

- El tamaño del lote puede tener un impacto significativo en el rendimiento del modelo y en el tiempo de entrenamiento.
- La principal ventaja de utilizar lotes de gran tamaño es que los aceleradores de hardware como las GPU pueden procesarlos de forma eficiente, por lo que el algoritmo de entrenamiento verá más instancias por segundo. Por ello, muchos investigadores y profesionales recomiendan utilizar el mayor tamaño de lote que quepa en la RAM de la GPU. Sin embargo, hay un inconveniente: en la práctica, los lotes grandes suelen provocar inestabilidades en el entrenamiento, sobre todo al principio, y el modelo resultante puede no generalizar tan bien como un modelo entrenado con lotes pequeños.

Tasa de aprendizaje, tamaño del lote y otros hiperparámetros

► Función de activación

- En general, la función de activación ReLU será un buen valor por defecto para todas las capas ocultas, pero para la capa de salida realmente depende de su tarea.

► Número de iteraciones

- En la mayoría de los casos, no es necesario ajustar el número de iteraciones de entrenamiento: basta con utilizar la parada anticipada.

- Para conocer más prácticas recomendadas sobre el ajuste de hiperparámetros de redes neuronales, consulta el excelente artículo de Leslie Smith de 2018.

Leslie N. Smith, "A Disciplined Approach to Neural Network Hyper-Parameters: Part 1—Learning Rate, Batch Size, Momentum, and Weight Decay", arXiv preprint arXiv:1803.09820 (2018).